

2nd Yr - Week 1 - Module

Topics to be covered: Language Used C and C++

1. SYNTAX

2. Conditions and loops

3. Programming problems

Introduction about C-Programming Language.

C-LANGUAGE

- * C is a general-purpose, statically-typed, procedural programming language developed by Dennis Ritchie at Bell labs in 1972.
- * It was originally designed to build the UNIX operating system.

Note: Today, it is widely considered as the foundation of modern computer science and software development.

Key features of C

- **PROCEDURAL LANGUAGE:** Code runs step-by-step using ordered functions.
- **FAST AND EFFICIENT:** Compiles directly into low level machine instructions.
- **DIRECT MEMORY ACCESS:** Uses pointers to control system resources directly.

- **HIGHLY PORTABLE**: code runs on various hardware with little modification.
- **RICH BUILT-IN LIBRARIES**: provides standard tools for math, strings and inputs.

STRUCTURE OF C-PROGRAM

#include <stdio.h> // 1. Preprocessor directive

// 2. Main function where execution starts

int main () {

// 3. Executable statement

printf ("Hello, world! \n");

// 4. Return status to the operating system

return 0;

}

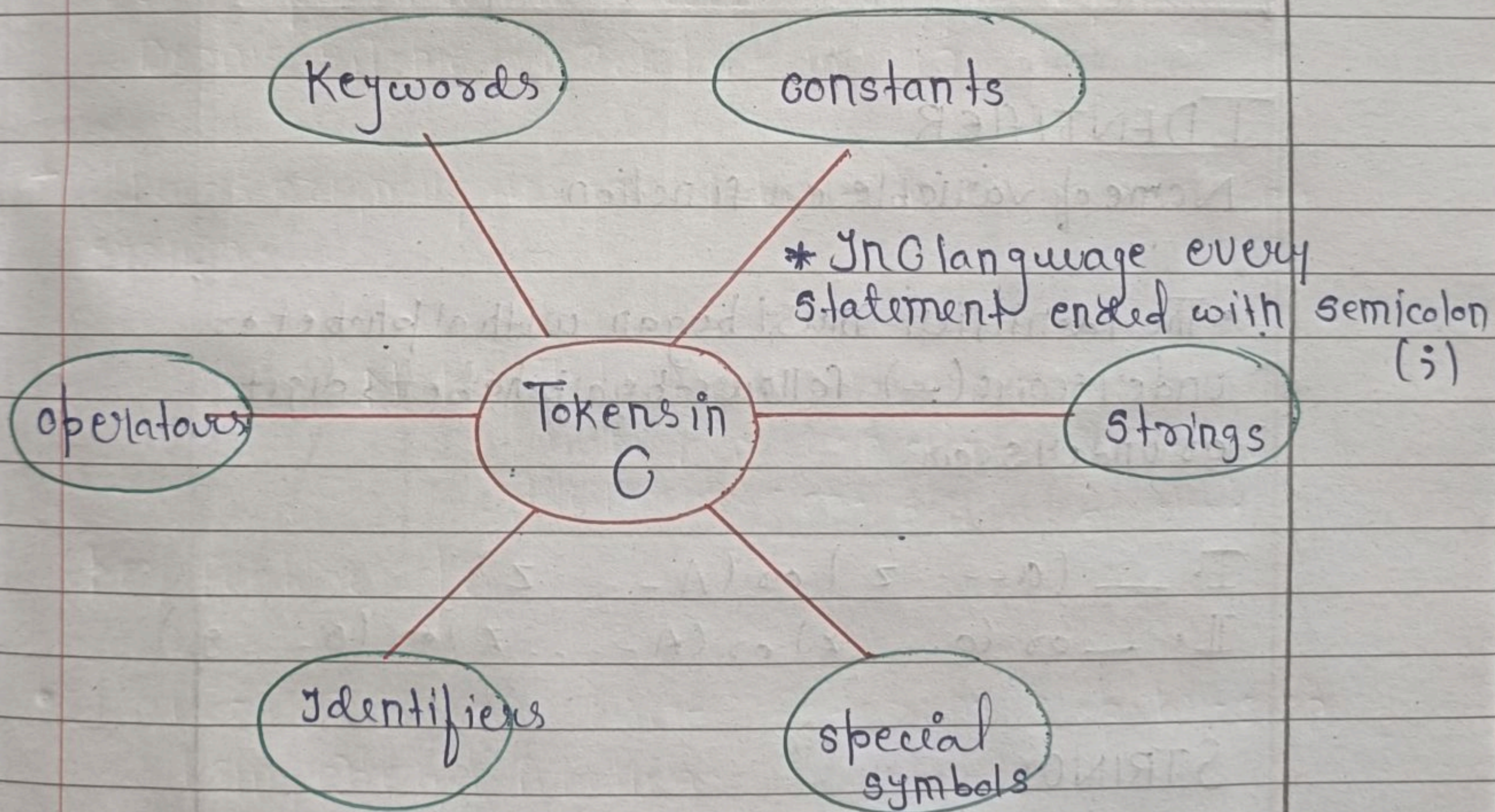
Code breakdown

- #include <stdio.h>: Instructs the preprocessor to load standard i/p / output functions.
- int main (): The primary entry point for any execution script.
- printf (): Displays text formatting out to your current console.

- Date Page No.
- return 0 ; : Tells the OS that the program completed successfully.

TOKEN IN C-LANGUAGE

In C programming, tokens are the smallest & meaningful units of code that the computer recognizes serving as the building blocks of a C-program. They include keywords, identifiers, constants, operators, special symbols and strings.



RESERVED KEYWORDS

Some keyword are reserved and can't used in C-language as a Name of variable.

RESERVED KEYWORDS

auto	double	<u>int</u>	struct
break	else	long	switch
case	enum	register	typedef
char	extern	<u>return</u>	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while.

IDENTIFIER

Name of variable or function

- The identifier must begin with alphabet or underscore (_) followed by alphabet, digit or underscore

I. (a - - - - z | or (A - - - - Z)

II. or (a - - - - x) or (A - - - - Z) or (0 - - - - 9)

STRING

Any sequence of character put in double quote.

CONSTANT

int a = 10;

↑

Integer constant

or int a = 10.1;

↑

decimal constant

* float constant

* character constant

* string constant

CONSTANT / LITERALS

Integer literal : 42

float literal : 3.14000

char literal : A

Double literal : 2.718280

String literal : Hello

OPERATOR:

Arithmetic operator : +, -, *, /, %

Unary operator : ++, --

Relational operator : >, <, >=, <=, !=, ==

Bitwise operator : &, ^, |, ~, >>, <<

logical operator : &&, ||, !

Ternary operator : ? :

int a = 10;

↓ Keyword ↓ operator ↓ constant

Identifier

special symbol

Any other symbol in program is special character.

int a = 10;

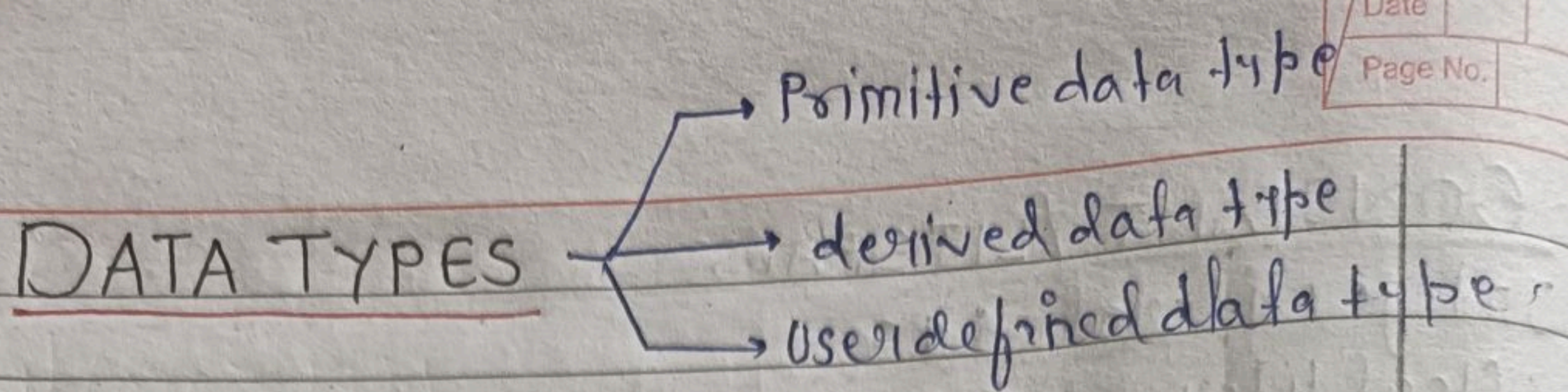
a = b + c;

↓ Identifier ↓ operator ↓ special symbol

• Identifier = 3

• Tokens = 6

DATA TYPES



Data type defines

1. Kind of value variable will hold
2. Amount of memory required
3. Interpretation of data

In C, a data type specifies the

- Kind of value a variable can hold
- compiler allocates memory and
- Interpretation

```
#include <stdio.h>
```

```
int main ( )
```

```
{
```

```
    int a = 3.14 ; // only Integer part is taken
```

```
    printf ("%d", &a);
```

```
    return 0;
```

```
}
```

• Output = 3

a = 3

STRUCTURE OF THE C-PROGRAM

	1	#include <stdio.h>	Header
	2	int main(void)	Main function
	3	{	
	4	// printf "Hello world"	Comment
BODY	5	printf ("Hello world");	statement
	6	return 0;	return
	7	}	

2. CONDITIONS AND LOOPS

In C programming, conditions and loops are the core control structures that manage the flow of execution.

- Conditions let your program make decisions, while loops let it repeat actions efficiently.

1. Conditional statements (Decision making)

Conditional structures check if an expression is true (non-zero) or false (zero) and direct the program down a specific path.

if statement

Executes a block of code only if the specified condition is true.

C

```
if (age > 18) {
```

```
    printf("You are an adult.\n");
```

```
}
```

- If-else statement

Provides an alternative block of code that runs if the initial condition is true.

C

```
if (score >= 50) {
```

```
    printf("passed\n");
```

```
    } else {
```

```
        printf("failed\n");
```

```
    }
```

- If else if ladder

Evaluates multiple expressions in sequence from top to bottom.

The first block with a true condition is executed.

C

```
if (grade >= 90) {
```

```
    printf("A\n");
```

```
    } else if (grade >= 80) {
```

```
        printf("B\n");
```

```
    } else if
```

```
        printf("C\n");
```

```
    }
```

Switch Statement

Matches an integer or character expression against predefined case constants. It acts as a cleaner alternative to long if-else chains.

C

```
eh(day) {
    case 1: printf("Monday\n"); break;
    case 2: printf("Tuesday\n"); break;
    default: printf("Invalid day\n");
}
```

Note: The break statement prevents the code from "falling through" into subsequent cases.

2. LOOPS (Repetitive Execution)

loop repeat a block of code until a termination condition evaluates to false.

for loop (Entry-controlled)

Best used when you know exactly how many times the code needs to run. It bundles initialization, condition checking, and incrementing/decrementing into one line.

C

```
for (int i = 0; i < 5; i++) {
    printf("%d", i);
}
```

y

output: 0 1 2 3 4

While loop (Entry-controlled)

Evaluates the condition before entering the loop body. Best used when the total number of iterations depends on dynamic criteria rather than a fixed count.

C

```
int count = 0;
while (count < 5) {
    printf("%d", count);
    count++;
}
```

do-while Loop (exit-controlled)

Evaluates the condition after executing the loop body. This guarantees that the internal block executes at least once, regardless of whether the initial condition is true or false.

C

```
int count = 5;
do {
    printf("%d", count);
    count++;
} while (count < 5);
```

3. LOOP Control statements

These keywords allow you to change the standard flow of execution within your loops.

- break: Immediately exists the innermost loop or switch statement.
- continue: Skips the remaining code inside the loop body for the current cycle and jumps straight to the next evaluation / iteration.

PROGRAMMING PROBLEMS

• C

1. Square

enter side length of the square 5

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

```
#include <stdio.h>
```

```
int main() {
```

```
    int side;
```

```
    printf("enter side length of the square");
```

```
    scanf("%d", &side);
```

```
    for (int i = 0; i < side; i++) {
```

```
        for (int j = 0; j < side; j++) {
```

```
            printf("* ");
```

```
        }
```

```
    }
```

```
    printf("\n");
```

```
}
```

```
return 0;
```

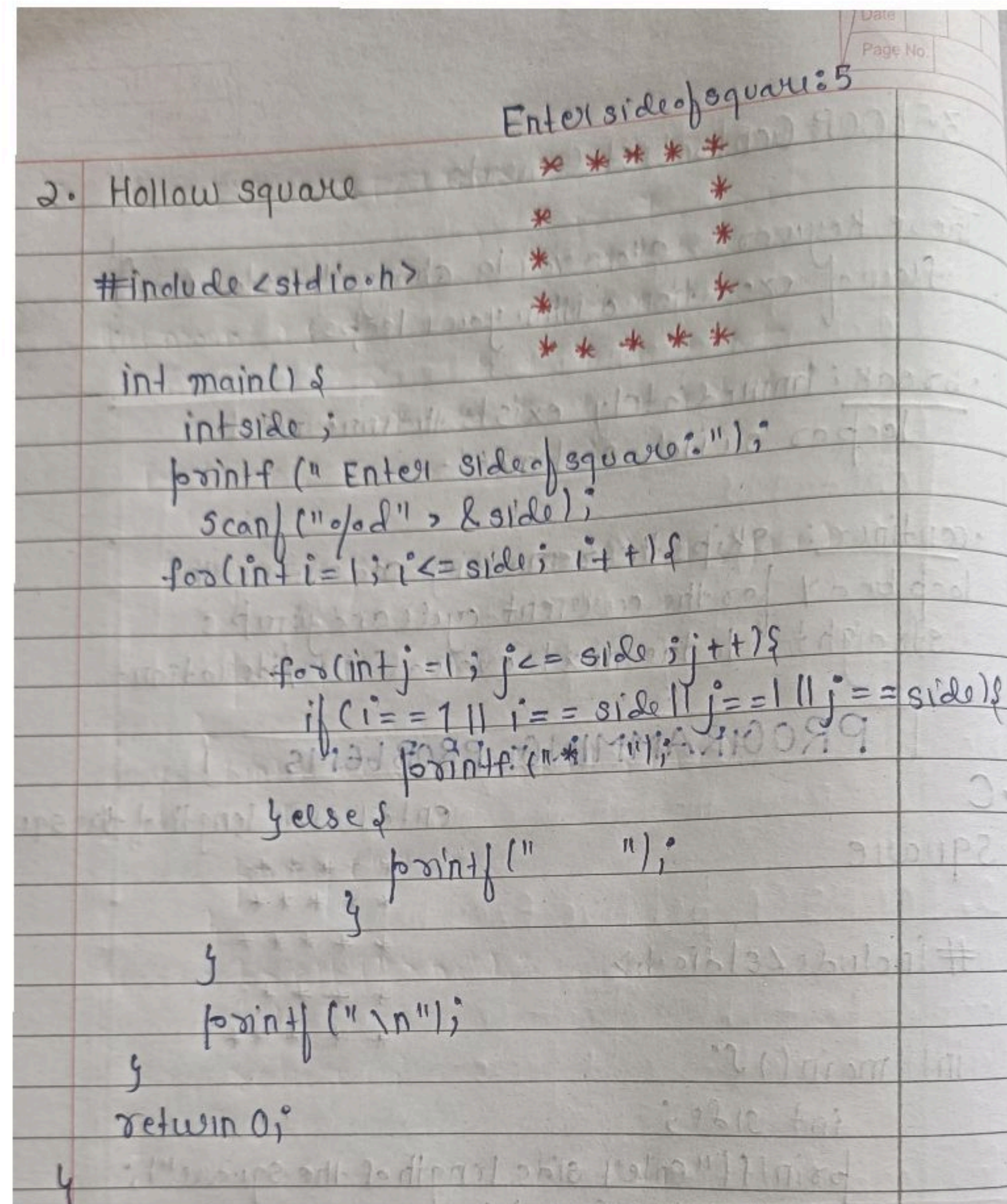
```
}
```

```
1
2 #include <stdio.h>
3
4 int main() {
5     int side;
6     printf("enter side length of the square ");
7     scanf("%d",&side);
8     for(int i=0;i<side;i++){
9         for(int j=0;j<side;j++){
10             printf("* ");
11
12         }
13         printf("\n");
14     }
15
16     return 0;
17 }
```

enter side length of the square 5

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

=== Code Execution Successful ===



```
1
2 #include <stdio.h>
3
4 int main() {
5     int side;
6     printf("Enter side of square : ");
7     scanf("%d", &side);
8     for(int i = 1; i <= side ; i++){
9
10         for(int j = 1 ; j <= side; j++){
11             if(i == 1 || i == side || j == 1 || j == side){
12                 printf("* ");
13
14             } else {
15                 printf(" ");
16
17             }
18         }
19
20         printf("\n");
21     }
22     return 0;
23 }
24
```

Enter side of square : 5

```
* * * * *
*       *
*       *
*       *
*       *
* * * * *
```

=== Code Execution Successful ===

3. TRIANGLE

```
#include <stdio.h>
```

```
int main() {
```

```
    int rows = 5;
```

```
    for (int i = 0; i < rows; i++) {
```

```
        for (int j = 0; j <= i; j++) {
```

```
            printf("* ");
```

```
        }
        printf("\n");
    }
```

```
    return 0;
```

```
}
```

Enter the no. of rows: 5

4. INVERTED TRIANGLE

```
#include <stdio.h>
```

```
int main() {
```

```
    int rows;
```

```
    printf("Enter the number of rows: ");
    scanf("%d", &rows);
```

```
    for (int i = rows; i >= 1; i--) {
```

```
        for (int j = 1; j <= i; j++) {
```

```
            printf("* ");
```

```
        }
```

```
        printf("\n");
```

```
    }
```

```
    return 0;
```

```
}
```

```
1
2 #include <stdio.h>
3
4 int main() {
5     int rows = 5;
6     for( int i = 0 ; i < rows ; i++){
7         for( int j = 0 ; j <= i ; j++){
8             printf("* ");
9
10        }
11        printf("\n");
12
13    }
14    return 0 ;
15 }
16
```

```
*
* *
* * *
* * * *
* * * * *
```

=== Code Execution Successful ===

```
1
2 #include <stdio.h>
3
4 int main() {
5     int rows ;
6
7     printf("Enter the number of rows: ");
8     scanf("%d", &rows);
9
10    for( int i = rows ; i >= 1 ; i--){
11        for( int j = 1 ; j <= i ; j++){
12            printf("* ");
13
14        }
15        printf("\n");
16
17    }
18    return 0 ;
19 }
20
```

Enter the number of rows: 5

```
* * * * *
* * * *
* * *
* *
*
```

*** Code Execution Successful ***

5. FULL PYRAMID PATTERN

```
#include <stdio.h>
```

```
int main() {
```

```
    int rows = 5;
```

```
    for (i = 0; i < rows; i++)
```

```
    {
        for (j = 0; j < 2 * (rows - i) + 1; j++)
```

```
        printf(" ");
```

```
    }
    for (int k = 0; k < 2 * i + 1; k++)
```

```
        printf("* ");
```

```
    }
    printf("\n");
```

```
    return 0;
```

```
}
```

6. INVERTED PYRAMID PATTERN

```
#include <stdio.h>
```

```
int main() {
```

```
    int rows;
```

```
    printf("Enter the number of rows: ");
    scanf("%d", &rows);
```

```
    for (int i = rows; i >= 1; i--) {
```

```
        for (int space = 0; space < rows - i; space++) {
```

```
            printf(" ");
```

```
        }
        for (int j = 1; j <= (2 * i - 1); j++) {
```

```
            printf("* ");
```

```
        }
        printf("\n");
```

```
    }
    return 0;
```

```
}
```

```
1
2 #include <stdio.h>
3
4 int main() {
5     int rows = 5 ;
6
7     for (int i = 0 ; i < rows ; i ++ ){
8         for (int j = 0 ; j < 2 * (rows - i) - 1 ; j ++){
9             printf(" ");
10
11         }
12         for(int k = 0 ; k < 2 * i + 1 ; k++)
13         {
14             printf("* ");
15
16         }
17         printf("\n");
18     }
19     return 0 ;
20 }
21
```

```

      *
    * * *
  * * * * *
* * * * * * *
* * * * * * * *
```

=== Code Execution Successful ===

```
1
2 #include <stdio.h>
3
4 int main() {
5     int rows = 5 ;
6
7     printf("Enter the number of rows :");
8     scanf("%d",&rows);
9
10    for(int i = rows ; i >= 1 ; i--){
11        for(int space = 0 ; space < rows - i ; space++ ){
12            printf(" ");
13        }
14        for(int j = 1 ; j <= (2*i-1) ;j++ ){
15            printf("*");
16
17        }
18        printf("\n");
19    }
20    return 0 ;
21 }
22
```

Enter the number of rows :5

*

=== Code Execution Successful ===

7. RIGHT HALF DIAMOND PATTERN

C.

```
#include <stdio.h>
```

```
int main() {
```

```
    int columns;
```

```
    printf("Enter the peak number of stars: ");
    scanf("%d", &columns);
```

```
    for(int i=1; i<=columns; i++) {
```

```
        for(int j=1; j<=i; j++) {
```

```
            printf("* ");
```

```
        }
        printf("\n");
```

```
    }
```

```
    for(int i=columns-1; i>=1; i--) {
```

```
        for(int j=1; j<=i; j++) {
```

```
            printf("* ");
```

```
        }
```

```
        printf("\n");
```

```
    }
```

```
    return 0;
```

```
}
```

*

```

1
2 #include <stdio.h>
3
4 int main() {
5     int columns ;
6
7     printf("Enter the peak number of stars :");
8     scanf("%d",&columns);
9
10    for(int i = 1 ; i <= columns ; i++){
11        for(int j = 1 ; j <= i ; j++ ){
12            printf("* ");
13        }
14
15        printf("\n");
16    }
17    for( int i = columns - 1 ; i >=1 ; i--){
18        for( int j = 1 ; j <= i ; j++){
19            printf("* ");
20        }
21        printf("\n");
22
23    }
24    return 0 ;
25 }
26

```

Enter the peak number of stars :5

```

*
* *
* * *
* * * *
* * * * *
* * * *
* * *
* *
*

```

=== Code Execution Successful ===

•C DIAMOND PATTERN

```

      * * *
    * * * *
  * * * * *
* * * * *
  * * * *
    * * *
      *
  
```

```
#include <stdio.h>
```

```
int main() {
```

```
    int rows, i, j;
```

```
    printf("enter number of rows for half diamond: ");
    scanf("%d", &rows);
```

```
    for (i = 1; i <= rows; i++) {
```

```
        for (j = 1; j <= rows - i; j++) {
```

```
            printf(" ");
```

```
        }
        for (j = 1; j <= (2 * i - 1); j++) {
```

```
            printf("*");
```

```
        }
```

```
        printf("\n");
```

```
    }
```

```
    for (i = rows - 1; i >= 1; i--) {
```

```
        for (j = 1; j <= rows - i; j++) {
```

```
            printf(" ");
```

```
        }
```

```
        for (j = 1; j <= (2 * i - 1); j++) {
```

```
            printf("*");
```

```
        }
```

```
        printf("\n");
```

```
    }
```

```
    return 0;
```

```
}
```

```

2  #include <stdio.h>
3
4  int main() {
5      int rows , i , j ;
6
7      printf(" enter number of rows for half diamond : ");
8      scanf("%d",&rows);
9
10     for(i=1;i<=rows;i++){
11         for(j = 1 ; j<=rows-i;j++){
12             printf(" ");
13
14         }
15         for(j=1;j<=(2*i-1);j++){
16             printf("*");
17
18         }
19         printf("\n");
20     }
21     for(i=rows-1;i>=1;i--){
22         for(j=1;j<=rows-i;j++){
23             printf(" ");
24
25         }
26         for(j=1;j<=(2*i-1);j++){

```

```

27
28         }
29         printf("\n");
30
31     }
32     return 0 ;
33 }

```

```

enter number of rows for half diamond : 5
*
***
*****
*****
*****
*****
***
*

```

=== Code Execution Successful ===

C++ LANGUAGE

C++ is a strongly-typed, case-sensitive language. Every standard program requires a `main()` function as its entry point, and statements must end with a semicolon (`;`).

1. SYNTAX.

```
#include <iostream> // Header library for input/output operations
```

```
int main() { // Execution begins here  
    // variable declaration : Type name = value;
```

```
    int age = 20;
```

```
    double price = 99.99;
```

```
    char grade = 'A';
```

```
    // output and input
```

```
    std::cout << "Age : " << age << std::endl;  
    // std::cin >> age;
```

```
    return 0; // signals successful program  
}             determination
```

2. CONDITIONAL STATEMENTS

Conditional statements evaluate boolean expressions (true or false) to control program flow.

Date _____
Page No. _____

- If-else ladder: Evaluates conditions sequentially.

- Switch block: compares a single variable against discrete constant values faster than long if-else chains for integers & characters.

C++

// 1. If-Else structure

```
if (age >= 18) {  
    std::cout << "Adult";  
}  
else if (age > 12) {  
    std::cout << "Teenager";  
}  
else {  
    std::cout << "child";  
}
```

// 2. Switch structure

```
Switch (grade) {  
    case 'A':  
        std::cout << "Excellent";  
        break;  
    case 'B':  
        std::cout << "Good";  
        break;  
    default:  
        std::cout << "Invalid grade";  
}
```

3. LOOPS (Iteration)

loops repeat a block of code while a specific condition remains true.

for loop

Best used when you know the exact number of iterations in advance.

CPP

```
// Syntax : for (initialization ; condition ; increment/decrement)
for (int i = 1 ; i <= 5 ; i++) {
    std::cout << i << " "; // output : 1 , 2 , 3 , 4 , 5
}
```

While loop

Best used when the number of iterations depends on a dynamic condition. It checks the condition before executing the body.

CPP

```
int energy = 3;
while (energy > 0) {
    std::cout << "Running... ";
    energy--; // crucial to prevent infinite loops
}
```

do-while loop

Guaranteed to execute at least once because the condition is evaluated at the bottom.

CPP

```
int choice = 0;
```

```
do {
```

```
    std::cout << "Menu printed at least once\n";
```

```
} while (choice != 0); // semicolon is mandatory
```

here.

LOOP - CONTROL STATEMENTS

- break: Exits the loop immediately

- continue: skips the rest of the current iteration and jumps directly to the next loop update

PROGRAMMING PROBLEMS IN C++

.C++

1. Hollow SQUARE

#include <iostream>

Using namespace std;

int main() {

int n = 5;

for(int i = 0; i < n; i++) {

for(int j = 0; j < n; j++) {

cout << " * ";

}

cout << "\n";

}

return 0;

}

2. Hollow SQUARE

#include <iostream>

Using namespace std;

int main() {

int n = 5;

for(int i = 0; i < n; i++) {

for(int j = 0; j < n; j++) {

if (i == 0 || i == n - 1 || j == 0 || j == n - 1) {

cout << " * ";

} else {

* * *

* * *

* * *

3. RIGHT HAND TRIANGLE

#include <iostream>

Using namespace std;

int main() {

int n = 5;

for(int i = 0; i < n; i++) {

for(int j = 1; j <= i; j++) {

cout << " * ";

}

cout << "\n";

}

return 0;

}

*

* *

* * *

* * * *

* * * * *

```
1 // Online C++ compiler to run C++ code online
2 #include <iostream>
3 using namespace std ;
4
5 int main() {
6     int n = 5 ;
7     for(int i = 0 ; i < n ; i++){
8         for(int j =0;j<n;j++){
9             cout << "* ";
10        }
11        cout<<"\n";
12    }
13
14
15    return 0;
16 }
```

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

=== Code Execution Successful ===

```

1
2 #include <iostream>
3 using namespace std ;
4
5 int main() {
6     int n = 5 ;
7     for(int i = 0 ; i < n ; i++){
8         for(int j = 0; j < n; j++){
9             if(i==0 || i==n-1 || j==0 || j==n-1){
10                 cout << "* ";
11             }else{
12                 cout << " ";
13             }
14         }
15         cout << "\n";
16     }
17     return 0 ;
18 }

```

Output

```

* * * * *
*       *
*       *
*       *
* * * * *

```

=== Code Execution Successful ===

main.cpp



Share

Run

Output

```
1
2 #include <iostream>
3 using namespace std ;
4
5 int main() {
6     int n = 5 ;
7     for(int i = 0 ; i <= n ; i++){
8         for(int j = 1; j <= i; j++){
9             cout << "* ";
10        }
11        cout << "\n";
12    }
13    return 0 ;
14 }
```

```
*
* *
* * *
* * * *
* * * * *
```

=== Code Execution Successful ===

4. INVERTED RIGHT HAND TRIANGLE

```

* * * * *
* * * *
* * *
* *
*
  
```

```

#include <iostream>
using namespace std;
  
```

```

int main() {
    int n = 5;
    for (int i = n; i >= 1; i--) {
        for (int j = 1; j <= i; j++) {
            cout << " * ";
        }
        cout << "\n";
    }
    return 0;
}
  
```

5. PYRAMID PATTERN

```

#include <iostream>
using namespace std;
  
```

```

int main() {
    int rows;
    cout << "enter number of rows:";
    cin >> rows;
  
```

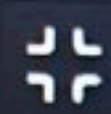
```

    for (int i = 1; i <= rows; i++) {
        for (int space = 1; space <= rows - i; space++) {
            cout << " ";
        }
        for (int j = 1; j <= (2 * i - 1); j++) {
            cout << " * ";
        }
        cout << endl;
    }
    return 0;
}
  
```

```

          *
         * *
        * * *
       * * * *
      * * * * *
     * * * * *
    * * * * *
  
```

main.cpp



Share

Run

Output

```
1
2 #include <iostream>
3 using namespace std ;
4
5 int main() {
6     int n = 5 ;
7     for(int i = n ; i >= 1 ; i--){
8         for(int j =1;j<=i;j++){
9             cout << "* ";
10        }
11        cout<<"\n";
12    }
13    return 0 ;
14 }
```

```
* * * * *
* * * *
* * *
* *
*
```

=== Code Execution Successful ===

```

1
2 #include <iostream>
3 using namespace std ;
4
5 int main() {
6     int rows ;
7
8     cout << "enter number of rows:";
9     cin >> rows ;
10
11     for(int i = 1 ; i <= rows; ++i){
12         for(int space = 1 ; space <= rows-i; ++space){
13             cout << " ";
14
15         }
16         for(int j = 1 ; j <= (2*i-1); ++j){
17             cout << "*";
18         }
19
20         cout << endl;
21     }
22     return 0 ;
23 }
24

```

enter number of rows:5

```

*
***
*****
*****
*****

```

=== Code Execution Successful ===

6. INVERTED PYRAMID PATTERN

```
#include <iostream>
using namespace std;
int main() {
    int rows;
```

```
    cout << "enter number of row:";
    cin >> rows;
```

```
    for (int i = rows; i >= 1; --i) {
        for (int space = 1; space <= rows - i; ++space)
            cout << " ";
```

```
        for (int j = 1; j <= (2 * i - 1); ++j) {
            cout << "*";
```

```
        }
        cout << endl;
```

```
    }
    return 0;
```

```
}
```

7. RIGHT HALF DIAMOND PATTERN

```

      *
     **
    ***
   ****
  *****
 *****
  ****
   ***
    **
     *
```

```
#include <iostream>
using namespace std;
```

```
int main() {
    int n;
```

```
    cout << "enter the maximum number of stars in Middle:";
    cin >> n;
```

```
    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= i; ++j) {
            cout << "*";
```

```
        }
        cout << endl;
```

```
    }
    for (int i = n - 1; i >= 1; --i) {
        for (int j = 1; j <= i; ++j) {
            cout << "*";
```

```
        }
        cout << endl;
```

```
}
```

```
return 0;
```

```
}
```

```

1
2 #include <iostream>
3 using namespace std ;
4
5 int main() {
6     int rows ;
7
8     cout << "enter number of rows:";
9     cin >> rows ;
10
11     for(int i = rows ; i>=1 ; --i){
12         for(int space = 1 ; space <=rows-i; ++space){
13             cout << " ";
14
15         }
16         for(int j = 1 ; j <=(2*i-1);++j){
17             cout << "*";
18         }
19
20         cout << endl;
21     }
22     return 0 ;
23 }
24

```

enter number of rows:5

*

I

=== Code Execution Successful ===



```
1
2 #include <iostream>
3 using namespace std ;
4
5 int main() {
6     int n ;
7
8     cout << "enter the maximun number of star in middle row:";
9     cin >> n ;
10
11     for(int i = 1 ; i<=n ; ++i){
12         for(int j = 1 ; j<=i; ++j){
13             cout << "*";
14         }
15
16         cout << endl;
17     }
18     for(int i =n-1;i>=1;--i){
19         for(int j = 1 ; j<=i; ++j){
20             cout<<"*";
21         }
22         cout<< endl;
23     }
24     return 0 ;
25 }
26
```

enter the maximun number of star in middle row:5

```
*
**
***
****
*****
****
***
**
*
```

=== Code Execution Successful ===

8. DIAMOND PATTERN

```
#include <iostream>
using namespace std;
```

```
int main() {
```

```
    int n;
```

```
    cout << "enter the number  
of rows for the first half: ";
```

```
    cin >> n;
```

```
    for (int i = 1; i <= n; ++i) {
```

```
        for (int space = 1; space <= n - i; ++space) {
```

```
            cout << "  ";
```

```
        }
        for (int j = 1; j <= (2 * i - 1); ++j) {
```

```
            cout << "* ";
```

```
        }
```

```
        cout << endl;
```

```
    }
```

```
    for (int i = n - 1; i >= 1; --i) {
```

```
        for (int space = 1; space <= n - i; ++space) {
```

```
            cout << "  ";
```

```
        }
```

```
        for (int j = 1; j <= (2 * i - 1); ++j) {
```

```
            cout << "* ";
```

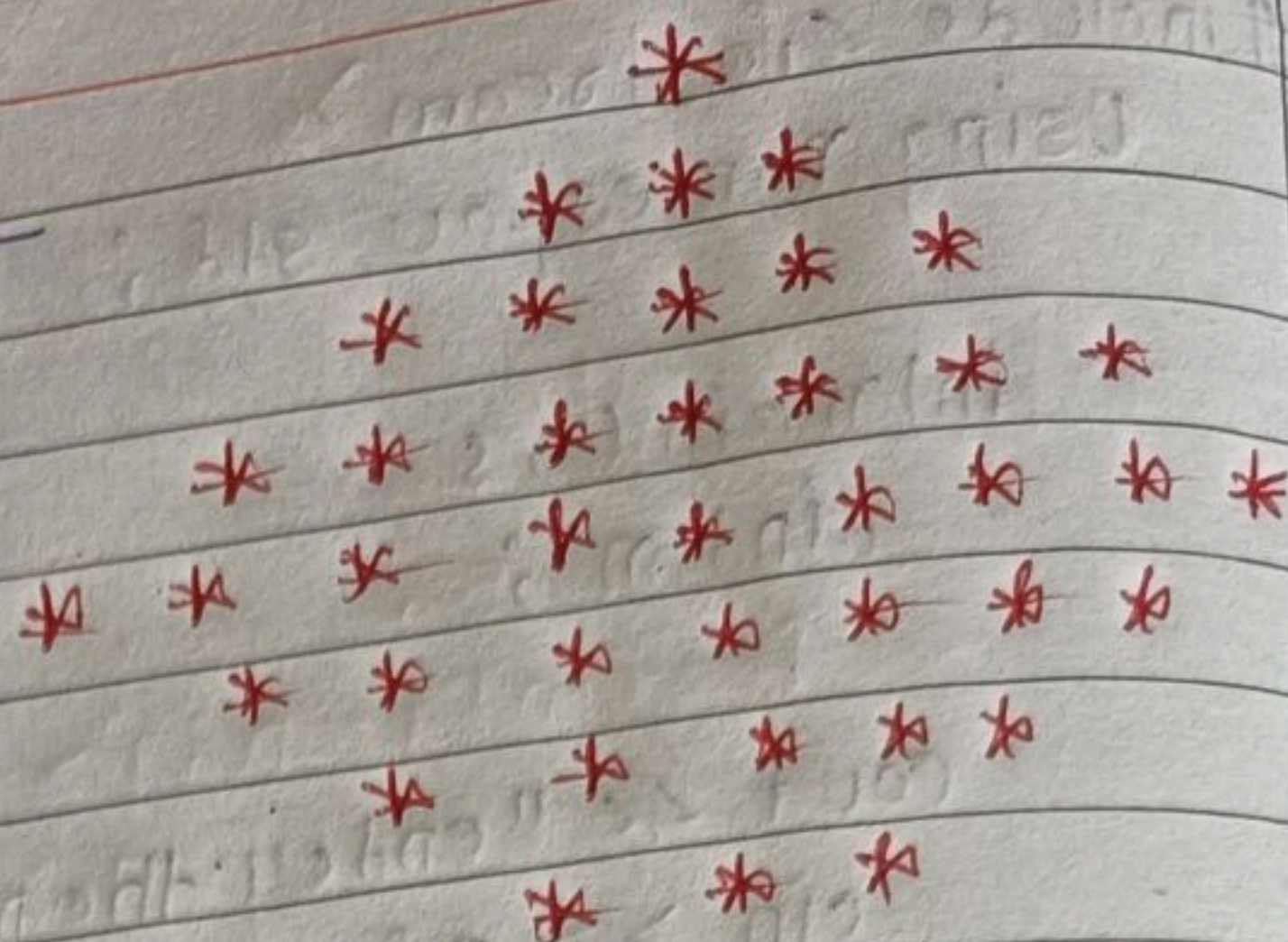
```
        }
```

```
        cout << endl;
```

```
    }
```

```
    return 0;
```

```
}
```



```

1
2 #include <iostream>
3 using namespace std ;
4
5 int main() {
6     int n ;
7
8     cout << "enter the number of rows for the first half:";
9     cin >> n ;
10
11     for(int i = 1 ; i<=n ; ++i){
12         for(int space = 1 ; space<=n-i; ++space){
13             cout << " ";
14         }
15
16         for(int j = 1 ; j <=(2*i-1);++j){
17             cout << "*";
18         }
19         cout << endl;
20     }
21     for(int i =n-1;i>=1;--i){
22         for(int space = 1 ; space<=n-i;++space){
23             cout<<" ";
24         }
25         for(int j = 1 ;j <= (2 * i-1) ; ++j ){
26             cout << "*";

```

```

27     }
28     cout<< endl;
29 }
30 return 0 ;
31 }
32

```

```

enter the number of rows for the first half:5
*
***
*****
*****
*****
*****
*****
***
*

```

=== Code Execution Successful ===